WheelScan

AI-Powered Accessibility Auditor for Public Spaces

Team : Alvira Parveen

B.Tech CSE (AI/ML) | 2026

The Problem We Are Solving

Understanding why WheelScan is needed in today's world

The Accessibility Crisis

* According to WHO, over 1 billion people (15% of the world's population) live with some form of disability. Many of them use wheelchairs.

* In India alone, over 2.68 crore people have physical disabilities (Census 2011). Most public buildings, roads, and spaces are not designed for them.

* There is no simple tool available for a common person to check whether a ramp is too steep, a door is too narrow, or a parking spot meets accessibility standards.

* Wheelchair users often arrive at a location only to find out it is not accessible. This wastes their time, energy, and causes frustration.

* Government accessibility guidelines exist (like RPWD Act 2016), but there is no easy way to check if places follow these rules.

* Currently, accessibility audits are done manually by professionals. This is expensive and slow. Most places never get audited at all.

1.3 Billion+

people with disabilities worldwide (WHO, 2023)

2.68 Crore

physically disabled people in India (Census)

75%

of public places in India lack basic wheelchair accessibility

0

mobile apps that scan a space and give instant accessibility score

Solution: WheelScan

A mobile app that turns your phone into an accessibility inspection tool

What does WheelScan do?

WheelScan lets any person take a photo of a public space (like a ramp, doorway, elevator, parking lot, staircase, road, or any area) and instantly get a detailed accessibility score out of 100. The app tells you what is good, what needs improvement, and gives specific recommendations. It also has a community feed where people share their audits, an accessibility map showing all audited locations, and a gamified profile with badges and streaks.

Scan Any Space

User opens the camera or uploads a photo from gallery. The app accepts any image of any public space. No special equipment needed ,just a smartphone.

Smart Space Recognition

After uploading, user either picks from 5 preset categories (Ramp, Doorway, Elevator, Parking, Staircase) OR types a custom description like 'road area near hospital'. The app understands 12+ space types.

Detailed Score & Breakdown

The app generates a score from 0 to 100. Each space is checked on multiple criteria. Every criterion gets a PASS (green), WARNING (orange), or CRITICAL (red) rating with a specific recommendation.

Interactive Accessibility Map

All audited locations appear on a map with color-coded pins. Green pins = score 75+, Orange = 50-74, Red = below 50. Users can filter by space type and tap pins to see details.

Community Feed

A social feed showing audits from all users. Each post shows the location, score, user name, description, and has like/comment/share buttons. Users can see what others have audited nearby.

Profile, Badges & Gamification

Users earn badges (First Scan, 5 Audits, Ramp Expert, etc.), maintain daily streaks, and see their audit history. Settings include edit profile, notifications, language, dark mode, and more.

How WheelScan Works

Complete user journey from opening the app to saving an audit

01

User Opens the App

The app starts with an animated splash screen showing the WheelScan logo. After 3 seconds, it automatically moves to the home dashboard which shows stats, recent audits, and a big 'Scan Now' button.

02

Takes Photo or Uploads Image

User taps 'Scan Now' and goes to the scan screen. They can either use the camera to take a new photo, or upload an existing photo from their gallery. The image_picker Flutter package handles this.

03

Selects Space Type

A beautiful bottom sheet slides up asking 'What type of space is this?' The user can pick from 5 preset types (Ramp, Doorway, Elevator, Parking, Staircase) OR tap 'Describe the Space' to type something custom like 'road area' or 'hospital corridor'.

04

App Analyzes the Space

A loading animation shows 'Analyzing Space...' for 2 seconds. Behind the scenes, the scoring engine matches the user's input to one of 12+ categories, picks the right accessibility criteria, and calculates a score.

05

Sees Detailed Results

The results screen shows: (1) An animated score ring that fills from 0 to the final score, (2) A PASS/WARNING/CRITICAL count, (3) Each issue with its severity badge and recommendation. The user can save the audit or go back and scan again.

Technology Stack

Every tool and package we used, and why we chose it

FRAMEWORK

Flutter 3.38.5

Flutter is Google's open-source UI toolkit for building apps from a single codebase. We chose it because our course is Flutter-based, it works on Android/iOS/Web, and has excellent widget system for building beautiful UIs.

LANGUAGE

Dart 3.10.4

Dart is the programming language used with Flutter. It supports async/await for handling operations like image picking, has strong typing for fewer bugs, and compiles to fast native code.

PACKAGE

image_picker ^1.1.2

This Flutter package allows users to take photos using the camera or pick existing images from their gallery. It works on both Android and web platforms. We use it in the scan screen.

UI SYSTEM

Material Design 3

Flutter's built-in material design system provides ready-made widgets like BottomNavigationBar, AlertDialog, BottomSheet, Switch, Scaffold, and more. We customized them with our green theme.

GRAPHICS

CustomPainter API

Flutter's canvas drawing API. We used it to create the animated score ring on the results screen (a circle that fills up based on the score) and the campus map grid with roads and building blocks.

ANIMATION

AnimationController

Flutter's animation system. We used it for: (1) Splash screen logo bounce and fade, (2) Score ring filling animation, (3) Staggered card reveal on results screen. Shows advanced Flutter knowledge.

AI/ML

Keyword-Based AI Engine

Our custom scoring service that matches user text descriptions to 12+ accessibility categories using keyword analysis. Each category has specific criteria rated as GOOD/ WARNING/CRITICAL. Explained in detail on the AI slide.

DEV TOOLS

VS Code + Chrome

We developed in VS Code with Flutter/Dart extensions and tested on Chrome browser for fast hot reload. The Android emulator (Pixel 8) was available for native testing.

Project Structure

How I organized the code ; clean, scalable, and professional

```
wheel_scan/lib/
  main.dart
  config/
    theme.dart
  models/
    audit_model.dart
  screens/
    splash_screen.dart
    home_screen.dart
    scan_screen.dart
    result_screen.dart
    map_screen.dart
    feed_screen.dart
    profile_screen.dart
  services/
    scoring_service.dart
  widgets/
    score_gauge.dart
```

What Each Folder Does & Why

main.dart — App Entry Point

This is the first file that runs when the app starts. It creates the MaterialApp, applies our custom green theme, and shows the SplashScreen. It also contains MainNavigation which controls the bottom tab bar and switches between all 5 main screens.

config/ — App Configuration

Contains theme.dart which defines ALL colors (primary green, accent blue, danger red, warning orange), text styles (heading, body, caption), gradients (hero, splash, shimmer), and reusable card decorations. Every screen imports colors from here so the app looks consistent.

models/ — Data Structures

Contains audit_model.dart which defines two classes: AuditResult (spaceType, overallScore, issues, imagePath, timestamp, locationName) and AuditIssue (title, description, severity, recommendation). These are the shapes of our data.

screens/ — All App Pages

Contains 7 screen files, one for each page the user can see. Each screen is either a StatelessWidget (no changing data) or StatefulWidget (has changing data like scan screen's isAnalyzing state). Each screen is a separate file for clean code organization.

services/ — Business Logic

Contains scoring_service.dart , the brain of the app. This file has the analyzeImage() function, all 12+ category definitions with their criteria, and the score calculation formula. Screens call services, they don't do heavy logic themselves.

widgets/ — Reusable Components

Contains small UI components that are used in multiple screens, like the score gauge widget. This avoids writing the same code twice. If we change a widget here, it updates everywhere it's used.

App Screens — Detailed Overview

Every screen explained , what it shows and what Flutter widgets power it

1. Splash Screen	Shows animated WheelScan logo bouncing in with elastic curve, title sliding up, tagline fading in, and a loading spinner. Background has green gradient with decorative circles for depth. After 3 seconds, smoothly fades to home screen using PageRouteBuilder.	`AnimationController, TickerProviderStateMixin, FadeTransition, SlideTransition, PageRouteBuilder, CurvedAnimation (elasticOut, easeOutCubic)`
2. Home Dashboard	Greeting with user name, green avatar, notification bell. Dark hero card with 'Scan Now' button and embedded stats (24 scans, 72 avg score, 12 places, 7 streak). Quick scan category chips. Impact cards (5 Issues Found, 3 Improved). Recent audit list with circular score badges. Bottom tip card linking to map.	`SafeArea, SingleChildScrollView, GestureDetector, Container with BoxDecoration, LinearGradient, BoxShadow, Row, Column, ListView (horizontal), Expanded`
3. Scan Screen	Two main options: 'Take a Photo' (camera) and 'Upload from Gallery'. After picking image, bottom sheet slides up with 5 preset space types + 'Describe the Space' custom option with text field and suggestion chips. Shows loading animation during analysis. Navigates to results.	`StatefulWidget, ImagePicker, showModalBottomSheet, showDialog, TextField, TextEditingController, Future.delayed, Navigator.push, async/await, mounted check`
4. Results Screen	Animated score ring drawn with CustomPainter fills from 0 to score over 1.5 seconds with gradient sweep and glowing dot. Gently pulses. Dark card with space type badge and PASS/WARNING/CRITICAL counts. Issue cards slide in one by one with staggered animation. Each card has severity icon, badge, description, and recommendation.	`CustomPainter, Canvas.drawArc, SweepGradient, AnimationController x3, TickerProviderStateMixin, CustomScrollView, SliverAppBar, SliverToBoxAdapter, Transform.translate, Opacity animation`
5. Map Screen	Simulated campus map drawn with CustomPainter (grid lines, road lines, building blocks). 8 color-coded location pins positioned with Stack + Positioned. Tapping a pin highlights it and shows name tooltip. Filter chips actually filter visible pins. Horizontally scrollable location cards at bottom sync with map pins.	`StatefulWidget, CustomPainter, Stack, Positioned, AnimatedContainer, ListView.builder (horizontal), GestureDetector, setState for filters and pin selection`
6. Community Feed	Header with filter icon. Horizontal filter chips (All, Ramps, Doorways, etc.). 6 audit cards from different users showing: avatar with initial, name, time ago, score badge with /100, location with pin icon, space type tag, description text, like/comment/share/bookmark buttons.	`SafeArea, Column, SizedBox with horizontal ListView, Expanded with vertical ListView, Container, CircleAvatar, Row with Expanded, Padding, Text styling`
7. Profile Screen	Green gradient avatar with initial. Stats row (24 Audits, 72 Avg Score, 12 Places, 5 Badges). Horizontal scrollable badges ; earned ones colored, locked ones grey. Tapping badges shows snackbar. 7-day streak card. Audit history with 6 entries. Settings: Edit Profile (dialog with text fields), Notifications (bottom sheet with toggles), Dark Mode (toggle), Language (selector with flags), Help & Support (bottom sheet), About WheelScan (dialog with app info), Log Out (confirmation dialog).	`StatefulWidget, setState, showDialog, showModalBottomSheet, Switch widget, TextEditingController, ScaffoldMessenger.showSnackBar, Navigator.pop, GestureDetector`

AI / ML Pipeline

How the image classification and scoring engine works , step by step

Complete AI Pipeline Flow

INPUT

User uploads image + selects space type OR types custom description

>

CLASSIFY

If preset: directly use selected category. If custom: run keyword matching against 12+ patterns (road, home, hospital, mall, transit, garden, office, etc.)

>

CRITERIA

Load the specific accessibility criteria for that category. Example: Ramp has 4 criteria (Handrails, Slope Angle, Surface Texture, Width). Parking has 3 (Accessible Sign, Space Width, Path to Entrance).

>

SCORE

Each criterion is rated: GOOD = 30 points, WARNING = 15 points, CRITICAL = 0 points. Formula: Score = (total points earned / maximum possible points) x 100. Clamped to 0-100 range.

>

OUTPUT

Final AuditResult object with: spaceType, overallScore (0-100), list of AuditIssue objects (each with title, description, severity, recommendation), imagePath, timestamp, locationName.

Keyword Matching Engine (for custom descriptions)

When user types a custom description instead of picking a preset category,

the engine converts text to lowercase and checks for keywords:

`'road', 'street', 'sidewalk', 'path'`	>> Road / Pathway
`'home', 'house', 'apartment', 'flat'`	>> Home / Residential
`'hospital', 'clinic', 'medical'`	>> Hospital / Medical
`'mall', 'shop', 'store', 'market'`	>> Mall / Commercial
`'bus', 'metro', 'station', 'transit'`	>> Transit / Bus Stop
`'garden', 'park', 'outdoor', 'field'`	>> Garden / Outdoor
`'office', 'lobby', 'building', 'corridor'`	>> Office / Building
`No match found`	>> Generic Accessibility Check

Score Calculation Formula

Each criterion gets a severity rating:

GOOD	= 30 points (meets standards)
WARNING	= 15 points (needs improvement)
CRITICAL	= 0 points (fails standard)

Formula:

```
Score = (Total Points / Max Points) x 100
```

Example: Ramp has 4 criteria. 3 GOOD + 1 WARNING = (90+15)/120 x 100 = 88

AI Scoring — All 12+ Categories

Every space type the app can analyze, with its specific accessibility criteria

Ramp
4 criteria

Handrails (both sides), Slope Angle (1:12 ratio), Surface Texture (non-slip), Width (>90cm)

Doorway
3 criteria

Door Width (min 81cm), Threshold (flush/ <1.3cm), Handle Type (lever preferred)

Elevator
3 criteria

Button Height (wheelchair reach), Door Width (adequate), Braille Labels (on buttons)

Parking
3 criteria

Accessible Sign (wheelchair symbol), Space Width (min 3.6m), Path to Entrance (clear route)

Staircase
3 criteria

Alternative Route (ramp/elevator nearby), Handrails (both sides), Tactile Warning (strips at top)

Road / Pathway
5 criteria

Surface Condition (even/paved), Curb Cuts (at crossings), Width (>1.2m), Obstacles (clear path), Tactile Paving

Home / Residential
4 criteria

Entrance Access (step-free), Door Width (>81cm), Floor Level (ground floor), Pathway to Door (clear/even)

Hospital / Medical
4 criteria

Main Entrance (automatic doors), Corridor Width (>1.8m for two wheelchairs), Signage (visible), Emergency Access

Mall / Commercial
4 criteria

Entrance (level access), Floor Navigation (>90cm aisles), Elevator (all public floors), Seating Areas (wheelchair spaces)

Transit / Bus Stop
4 criteria

Platform Level (matches vehicle), Waiting Area (wheelchair space), Approach Path (accessible), Info Display (1.2m height)

Garden / Outdoor
4 criteria

Path Surface (paved/compacted), Path Width (>1.2m), Slope (max 1:20), Rest Areas (benches every 50m)

Office / Building
4 criteria

Entry Door (automatic/easy-open), Corridor Width (>1.5m), Floor Surface (non-slip), Reception Height (76cm lowered section)

AI Recommendation Agent — How It Works

How the AI Action Plan is generated, step by step (built with OpenAI Codex)

Recommendation Agent Pipeline

INPUT

Existing scoring engine output: space type + every criterion rated GOOD / WARNING / CRITICAL

>>

READ ISSUES

Agent reads each flagged WARNING/CRITICAL criterion with the space category context — nothing is scored again, only read

>>

DRAFT

For each flagged issue, drafts why it matters, a specific fix, and the accessibility standard it is based on

>>

SELF-CHECK

Agent reviews its own draft, rewrites anything vague, and adds a confidence flag plus cost and effort tags

>>

RANKED PLAN

Final AI Action Plan: prioritized recommendations with a visible self-check note, ready to display or copy

What Each Recommendation Includes

Every flagged issue gets a full set of fields,

not just a label, so the user always knows what to do next:

`Why it matters`	>> Plain-language impact explanation
`Recommended fix`	>> Specific, concrete action to take
`Standard reference`	>> The accessibility guideline behind it
`Priority rank`	>> Ordered by importance for access
`Impact & effort labels`	>> e.g. High access impact / Medium effort
`Cost tag`	>> Low (DIY) / Medium / High (structural)
`Confidence flag`	>> High confidence vs. Verify on-site
`How to verify`	>> The exact post-fix check to confirm it worked

Agent Self-Check Discipline

Before any recommendation is shown, the agent checks it is:

Specific	not a restatement of the score itself
Evidence-based	references a real accessibility standard
Honest	flags uncertainty instead of overstating it

Visible proof:

`A self-check note is shown on every card`

"Draft generated from CRITICAL criterion and checked for actionability."

The AI Action Plan

Real example output , from a photo scan to a prioritized fix list

From score to action, in one scan

For this hackathon, WheelScan's existing accessibility score was extended with an AI Recommendation Agent, built with OpenAI Codex. For every WARNING or CRITICAL issue the scoring engine flags, the agent now generates a specific fix, a priority rank, impact/effort/cost tags, a confidence flag, and a visible self-check note — turning a plain score into a real action plan.

Alternative Route — Critical

No ramp or elevator found nearby. Provide a step-free alternative route and add clear signage at the staircase.

Handrails — Warning

Handrail present on one side only. Install continuous handrails on both sides of the stair flight.

Tactile Warning — Warning

No tactile strips at the top of the stairs. Add tactile warning tiles before the first step for low-vision users.

Impact & Effort Tags

Each fix is labeled by access impact and effort required, so the most important fix is obvious at a glance.

Cost & Confidence Flags

A Low/Medium/High cost tag plus a High Confidence vs. Verify-on-site flag keep every recommendation honest.

Agent Self-Check

Every card shows a visible note confirming the recommendation was reviewed for actionability before being shown.

Flutter Concepts We Used

Every major Flutter concept in the app , explained in simple words for viva

StatefulWidget vs StatelessWidget

StatelessWidget is for screens that don't change (like Feed — it always shows the same data). StatefulWidget is for screens where something changes (like Scan — the isAnalyzing variable toggles between showing the scan view and the loading view). When data changes, StatefulWidget rebuilds the UI.

Navigator.push() and Navigator.pop()

push() adds a new screen on top of the current one (like going from Scan to Results). pop() removes the top screen and goes back. pushReplacement() replaces the current screen (used in splash to go to home without a back button).

CustomPainter

Flutter's canvas drawing API. We draw directly on the screen pixel by pixel. Used for: (1) The score ring — we draw an arc that fills based on score percentage, (2) The map grid — we draw lines, rectangles for roads and buildings. Shows advanced Flutter knowledge.

BottomSheet + AlertDialog

BottomSheet slides up from the bottom (used for space type selector, notifications settings, language picker). AlertDialog pops up in the center (used for edit profile, about app, logout confirmation). Both are modal — they block the screen behind.

SingleChildScrollView + ListView

SingleChildScrollView makes a Column scrollable (used on Home, Profile). ListView.builder is for long lists — it only builds the items visible on screen, making it memory efficient (used on Feed). Without scrolling, content would overflow on small screens.

setState()

This is the function that tells Flutter 'hey, something changed, please rebuild the screen'. Example: when user taps a filter on the Map screen, we call setState() to update _selectedFilter, and Flutter redraws only the pins that match the new filter.

async / await / Future

These handle operations that take time. When we pick an image, the phone needs time to open the gallery. 'await' pauses the code until the image is picked. 'Future.delayed' creates an artificial wait (we use 2 seconds to simulate AI processing).

AnimationController + CurvedAnimation

AnimationController controls the timing (how long, when to start). CurvedAnimation adds easing (elasticOut makes the logo bounce, easeOutCubic makes the score ring slow down at the end). We chain multiple controllers for staggered effects.

GestureDetector + onTap

GestureDetector wraps any widget to make it tappable. We use it on cards, buttons, map pins, badges, settings items, and more. Without it, containers and text are not interactive. Every interactive element in the app uses GestureDetector.

image_picker Package

External Flutter package (added in pubspec.yaml). Provides ImagePicker class with pickImage() method. Supports two sources: ImageSource.camera (opens camera) and ImageSource.gallery (opens photo picker). Returns an XFile with the image path.

Live Demo Plan

Exact demo script , what to show, in what order, and what to say

1

App Launch (30 sec)

Open the app. Show the animated splash screen — logo bounces in, title slides up, tagline fades in. After 3 seconds it automatically transitions to the home dashboard. Point out the smooth fade transition.

2

Home Dashboard (1 min)

Walk through: greeting with your name, green avatar, notification bell (tap it — shows snackbar). Dark hero card with stats. Tap 'Scan Now' — it goes to scan tab. Come back. Show quick scan categories, impact cards, recent audits with colored scores, bottom tip card.

3

Scan with Preset Type (1.5 min)

Go to Scan tab. Tap 'Upload from Gallery'. Pick a photo of a RAMP from campus. Bottom sheet appears — tap 'Ramp'. Watch the 2-second analyzing animation. Results screen appears with animated score ring filling up. Walk through each issue card.

4

Scan with Custom Description (1.5 min)

Go back. Upload a photo of a ROAD or PARKING area. This time, tap 'Describe the Space'. Type 'road area near campus' or tap the suggestion chip. Hit Analyze. Show that the results now say 'Road / Pathway' with relevant criteria like curb cuts and surface condition.

5

Map Screen (45 sec)

Switch to Map tab. Show the campus map with colored pins. Tap a pin — name tooltip appears. Tap filter chips — pins filter by type. Scroll the location cards at bottom. Tap a card — the matching pin highlights on the map.

6

Community Feed (30 sec)

Switch to Feed tab. Scroll through community audits. Point out different users, different scores (green/orange/red), location names, descriptions, like and comment counts. Show filter chips at top.

7

Profile & Settings (1 min)

Switch to Profile tab. Show your stats, earned badges (tap one — shows snackbar), 7-day streak. Scroll to settings. Tap Edit Profile — show dialog. Tap Language — show selector with flags. Tap About WheelScan — show app info with your name. Tap Log Out — show confirmation.

8

Closing Statement (30 sec)

Summarize: 'WheelScan demonstrates 7 screens, 12+ space categories, animated UI, and a keyword-based AI engine — all built in Flutter/Dart. In production, we would add a real TFLite model, Firebase backend, and Google Maps.'

Future Scope

What we would add in the production version with more time and resources

Real AI Model with TFLite

Currently, the app uses keyword-based scoring (user tells the app what type of space it is). In production, we would train a MobileNetV2 CNN model on thousands of accessibility images using Google Colab. The model would be exported as a .tflite file and deployed on-device using the tflite_flutter package. The AI would automatically classify what it sees in the photo — no user input needed.

Firebase Backend

Right now, all data is hardcoded (dummy stats, dummy feed). With Firebase, we would add: (1) Firebase Auth for user login with email/Google, (2) Cloud Firestore to store all audit data in the cloud, (3) Firebase Storage to save photos, (4) Real-time streams so the feed updates live when anyone posts a new audit.

Google Maps Integration

Replace our simulated map (drawn with CustomPainter) with the real Google Maps SDK. Show actual GPS coordinates, calculate distances, provide turn-by-turn accessible routes, and display real location names using geocoding. Requires a Google Maps API key.

PDF Report Export

Generate professional accessibility audit reports in PDF format using the Flutter 'pdf' package. Each report would include the scanned photo, location details, overall score, each issue with its rating, and all recommendations. Facility managers could use these reports to plan improvements.

Voice Input with speech_to_text

Let users describe spaces using voice instead of typing. The Flutter speech_to_text package converts speech to text in real time. This makes the app more accessible for users who have difficulty typing, which aligns with our mission of accessibility for everyone.

Multi-Language Support

Full translation into Hindi, Spanish, French, and Arabic using Flutter's internationalization (i18n) system. The language selector in Profile settings already exists — we just need to add translation files. This would make WheelScan a global accessibility tool.

Thank You!

WheelScan

Making spaces accessible for everyone

Team: Alvira Parveen

B.Tech CSE (AI/ML)

Built with Flutter 3.38 & Dart 3.10 | July 2026